

CityReport

Σύστημα Αναφοράς & Διαχείρισης Προβλημάτων Πόλης

Η Γλώσσα Προγραμματισμού Python | Π.Μ.Σ. Πληροφοριακά Συστήματα & Υπηρεσίες | 2025-2026

Πανεπιστήμιο Πειραιώς | Τμήμα Ψηφιακών Συστημάτων

Το Πρόβλημα & ο Στόχος

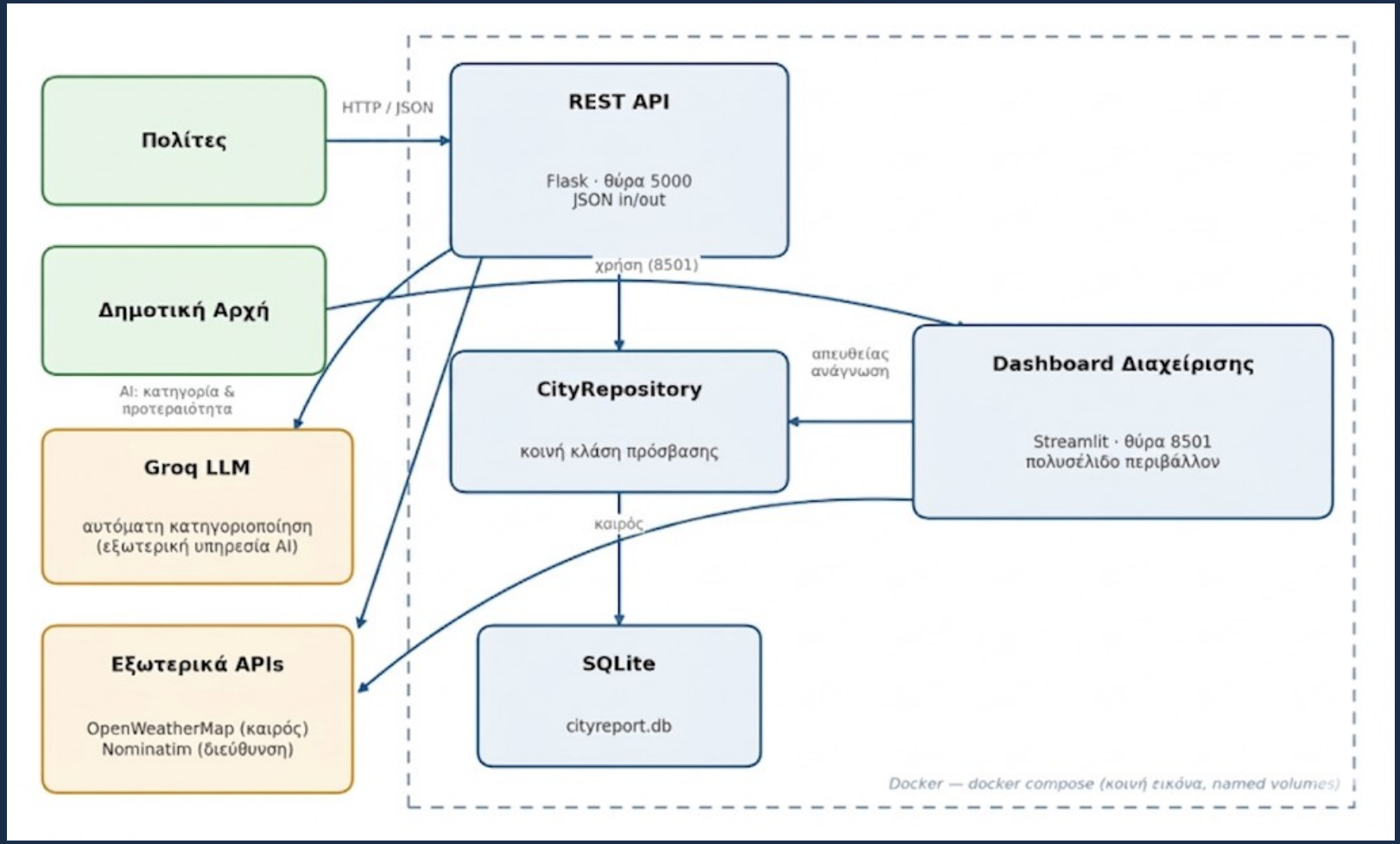
Το Πρόβλημα

- Οι πολίτες δεν έχουν εύκολο τρόπο να αναφέρουν προβλήματα στην πόλη
- Λακκούβες, χαλασμένα φώτα, εγκαταλελειμμένα οχήματα, σκουπίδια
- Οι αρχές δεν έχουν κεντρική εικόνα των προβλημάτων
- Δεν υπάρχει παρακολούθηση της πορείας επίλυσης

Ο Στόχος

- Ανώνυμη υποβολή αναφορών από πολίτες μέσω REST API
- Παρακολούθηση πορείας με μοναδικό Ticket ID
- Κεντρικό περιβάλλον διαχείρισης για τους αρμόδιους
- Αυτόματη κατηγοριοποίηση με Τεχνητή Νοημοσύνη
- Στατιστικά & οπτικοποιήσεις για λήψη αποφάσεων

Αρχιτεκτονική Συστήματος



Εργαλεία & Τεχνολογίες

🐍 Python 3

Γλώσσα υλοποίησης

⚗️ Flask

REST API Backend

📊 Streamlit

Admin Dashboard

🗄️ SQLite

Βάση Δεδομένων

🤖 Groq LLM

Τεχνητή Νοημοσύνη

📈 pandas / Plotly

Ανάλυση & Οπτικοποίηση

🌤️ OpenWeatherMap

Καιρός (Εξωτερικό API)

📍 Nominatim

Geocoding (Εξωτερικό API)

🐳 Docker

Containerization

Δομή Project

Backend

run.py

Εκκίνηση Flask server

src/api/routes.py

6 REST endpoints, AI, geocoding

app/repository.py

CityRepository - όλες οι SQL πράξεις

app/model/problem.py

Κλάση Problem (model)

src/database/db.py

Δημιουργία πινάκων SQLite

Dashboard

dashboard.py

Εκκίνηση Streamlit & navigation

pages/1_Home.py

KPIs & γραφήματα γρήγορης εικόνας

pages/2_Reports.py

Φίλτρα, αναλυτική προβολή, καιρός

pages/3_Statistics.py

Στατιστικά, χάρτης, χρονολόγιο

Config & Data

seed_data.py

22 αναφορές + placeholder εικόνες

seed_via_api.py

Μαζική υποβολή μέσω REST API

docker_init.py

Αρχικοποίηση DB στο Docker

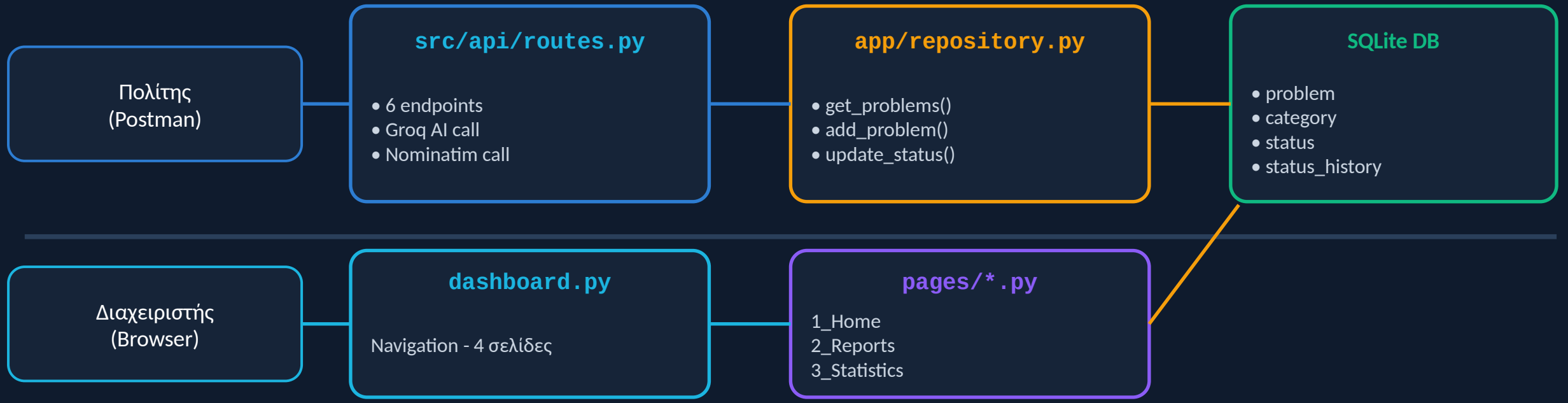
docker-compose.yml

2 services: api + dashboard

.env

GROQ_API_KEY, OWM_API_KEY

Ροή Κώδικα - Πώς Συνδέονται τα Αρχεία



Εξωτερικές Υπηρεσίες & Υποδομή

🤖 Groq API

routes.py → AI κατηγοριοποίηση

🌤️ OpenWeatherMap

2_Reports.py → καιρός

📍 Nominatim

routes.py → διεύθυνση (αποθηκεύεται στη βάση)

🐳 Docker

docker-compose.yml → api + dashboard

Μοντέλο Δεδομένων - Κλάση Problem

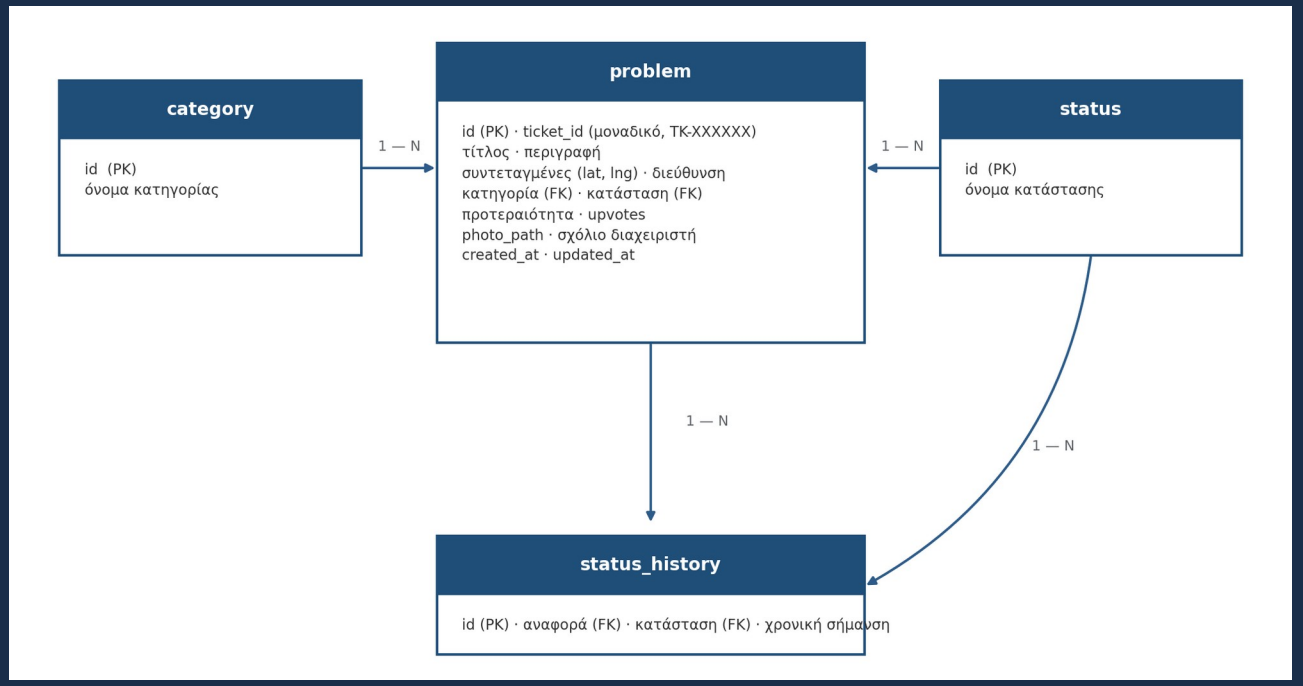
Τι είναι η κλάση Problem;

- Αναπαριστά μια αναφορά προβλήματος στη μνήμη
- Κάθε νέα αναφορά → νέο αντικείμενο Problem
- Δημιουργεί αυτόματα μοναδικό Ticket ID (TK-XXXXXX) με uuid
- Μέθοδος to_dict() → μετατρέπει σε JSON για το API
- Χρησιμοποιείται από το API και το repository

app/model/problem.py

`ticket_id`	→ TK-A1B2C3 (αυτόματο)
`title / description`	→ από τον πολίτη
`latitude / longitude`	→ συντεταγμένες
`category_id`	→ από AI (1-5)
`priority`	→ Low / Medium / High (AI)
`status_id`	→ 1=Αναφέρθηκε (default)
`photo_path`	→ διαδρομή αρχείου
`upvotes`	→ μετρητής υποστήριξης

Βάση Δεδομένων - SQLite



4 Πίνακες

problem

Κεντρικός πίνακας αναφορών

category

5 κατηγορίες (lookup)

status

4 καταστάσεις (lookup)

status_history

Ιστορικό αλλαγών - για χρόνο επίλυσης

CityRepository - Η Γέφυρα προς τη Βάση

Γιατί;

- Όλη η SQL σε ένα σημείο στον κώδικα
- Το API και το Dashboard χρησιμοποιούν την ίδια κλάση
- Αν αλλάξει η βάση → αλλάζει μόνο το repository
- Repository Pattern - γνωστό αρχιτεκτονικό πρότυπο

```
REST API   →  CityRepository  →  SQLite

Dashboard  →  CityRepository  →  SQLite
```

Κοινή κλάση - επαναχρησιμοποίηση κώδικα

app/repository.py

`get_problems()`	Φέρνει όλες τις αναφορές
`add_problem(problem)`	Αποθηκεύει νέα αναφορά
`update_problem_status()`	Αλλάζει κατάσταση + γράφει history
`find_duplicate()`	Ελέγχει αν υπάρχει ίδια αναφορά
`upvote_problem()`	Αυξάνει upvotes κατά 1
`update_photo_path()`	Αποθηκεύει διαδρομή φωτογραφίας

REST API - Κανάλι Πολιτών

POST	`/api/problems`	Υποβολή νέας αναφοράς + AI κατηγοριοποίηση → ticket_id
GET	`/api/problems`	Αναζήτηση & φιλτράρισμα (κατηγορία, κατάσταση, ημερομηνία)
GET	`/api/problems/<ticket_id>`	Παρακολούθηση πορείας αναφοράς
PUT	`/api/problems/<ticket_id>`	Ενημέρωση περιγραφής από πολίτη
POST	`/api/problems/<ticket_id>/photo`	Ανέβασμα φωτογραφίας (multipart/form-data)
POST	`/api/problems/<ticket_id>/upvote`	Υποστήριξη αναφοράς + ανίχνευση διπλοτύπων

POST /api/problems - Τι Γίνεται Μέσα;

1	**Validation**	Έλεγχος ότι υπάρχουν title, description, latitude, longitude → αλλιώς HTTP 400
2	**Duplicate Check**	Υπάρχει ίδια αναφορά κοντά; → αν ναι, upvote και επιστροφή ticket_id
3	**Geocoding (Nominatim)**	Συντεταγμένες → αναγνώσιμη διεύθυνση (π.χ. 'Λεωφ. Συγγρού 150, Αθήνα')
4	**AI Κατηγοριοποίηση (Groq)**	Τίτλος + περιγραφή → LLM → category_id (1-5) + priority (Low/Medium/High)
5	**Αποθήκευση**	Νέο αντικείμενο Problem → CityRepository.add_problem() → SQLite
6	**Απόκριση**	Επιστροφή ticket_id + auto_category + auto_priority με HTTP 201

AI Υποσύστημα - Groq & Llama

🤖 Πώς δουλεύει;

- Ο πολίτης γράφει απλά την περιγραφή
- Στέλνουμε τίτλο + περιγραφή στο Groq API
- Το μοντέλο llama-3.3-70b-versatile διαβάζει και αποφασίζει
- Επιστρέφει JSON με category_id και priority
- Αν αποτύχει → fallback: κατηγορία 'Άλλο', priority 'Medium'

```
response_format: {"type": "json_object"}
```

Αναγκάζουμε το μοντέλο να επιστρέφει πάντα έγκυρο JSON

Παράδειγμα

📥 Είσοδος (από πολίτη):

```
title: "Σπασμένος αγωγός νερού"
description: "Πλημμύρισε το πεζοδρόμιο"
```

📤 Έξοδος (από AI):

```
{ "category_id": 5, "priority": "High" }
```

✅ Αποτέλεσμα:

- Κατηγορία 'Άλλο' (id=5)
- Προτεραιότητα 'High'
- Αποθηκεύεται αυτόματα στη βάση

Admin Dashboard - Streamlit

🏠 `1_Home.py`	KPIs: σύνολο, εκκρεμή, σε εξέλιξη, επιλυμένα. Γραφήματα κατανομής ανά κατηγορία & κατάσταση.
📋 `2_Reports.py`	Φίλτρα, αναλυτική προβολή, φωτογραφία, καιρός OpenWeatherMap, διεύθυνση Nominatim, αλλαγή κατάστασης.
📊 `3_Statistics.py`	Κατανομές (pie/bar), χρόνοι επίλυσης από status_history, χάρτης, χρονολόγιο. Εξαγωγή CSV.

Διαβάζει απευθείας τη βάση μέσω CityRepository - χωρίς REST API

Υποσυστήματα

🌤️ Εξωτερικά APIs

- OpenWeatherMap: καιρός στην τοποθεσία αναφοράς (2_Reports.py)
- Nominatim (GeoPy): συντεταγμένες → διεύθυνση (routes.py + 2_Reports.py)
- Timeout 10sec & error handling — αν αποτύχει δεν κρασάρει το σύστημα

📈 Ανάλυση Δεδομένων

- pandas: φόρτωση δεδομένων από SQLite σε DataFrame, ομαδοποιήσεις
- Plotly: διαδραστικά γραφήματα (pie, bar, line, scatter_map)
- Μέσος χρόνος επίλυσης: υπολογισμός από status_history με χρονικές πράξεις

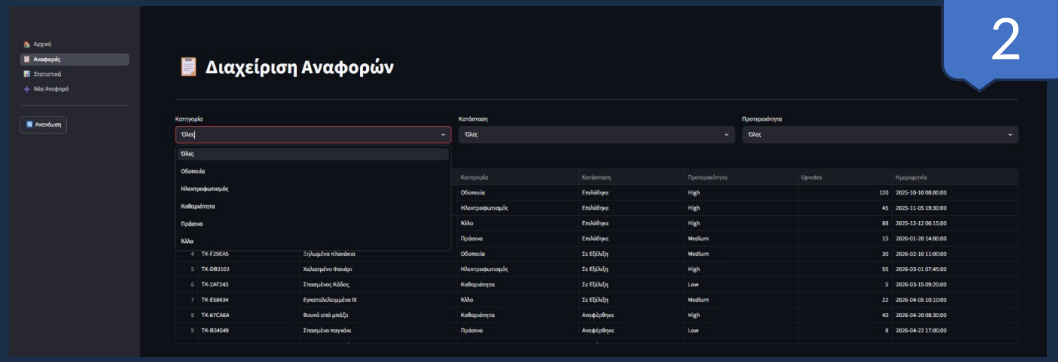
🐳 Docker

- 2 services: cityreport_api (5000) + cityreport_dashboard (8501)
- Κοινή εικόνα — shared named volumes για DB και uploads/
- docker_init.py: αρχικοποίηση βάσης + seed μόνο αν είναι κενή

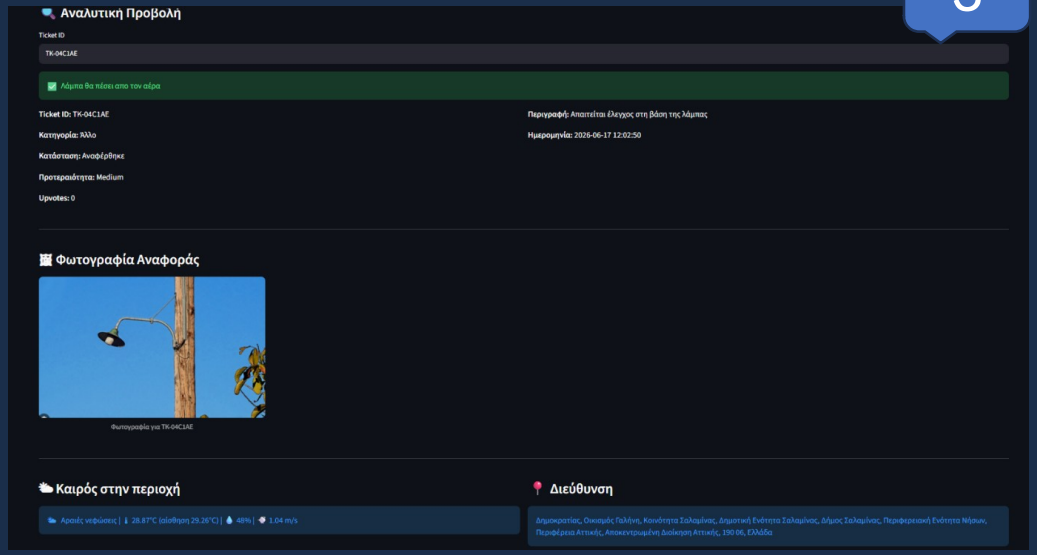
Αποτελέσματα - Dashboard



1



2



3

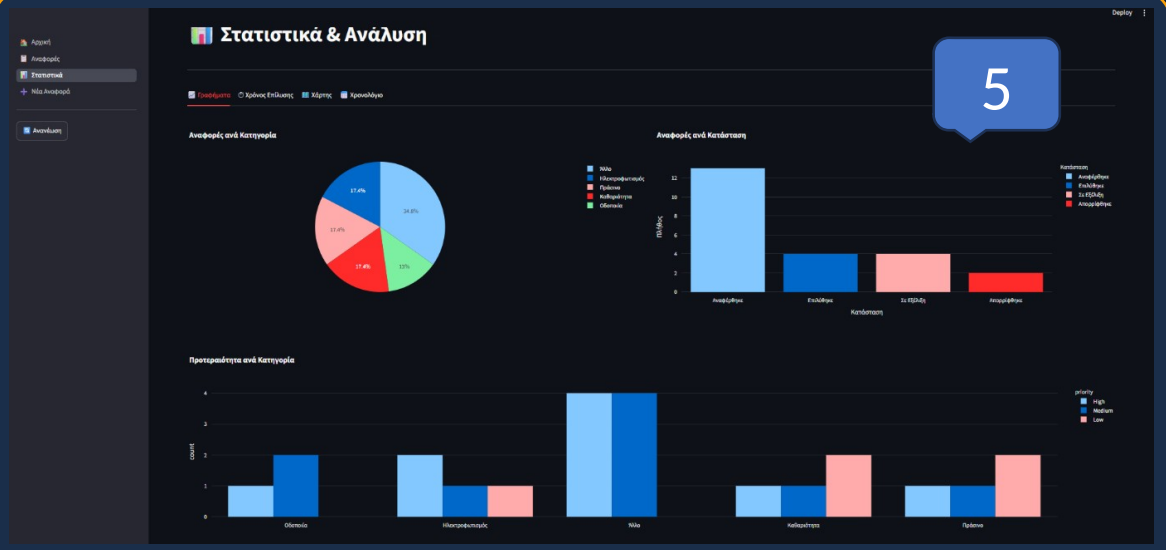


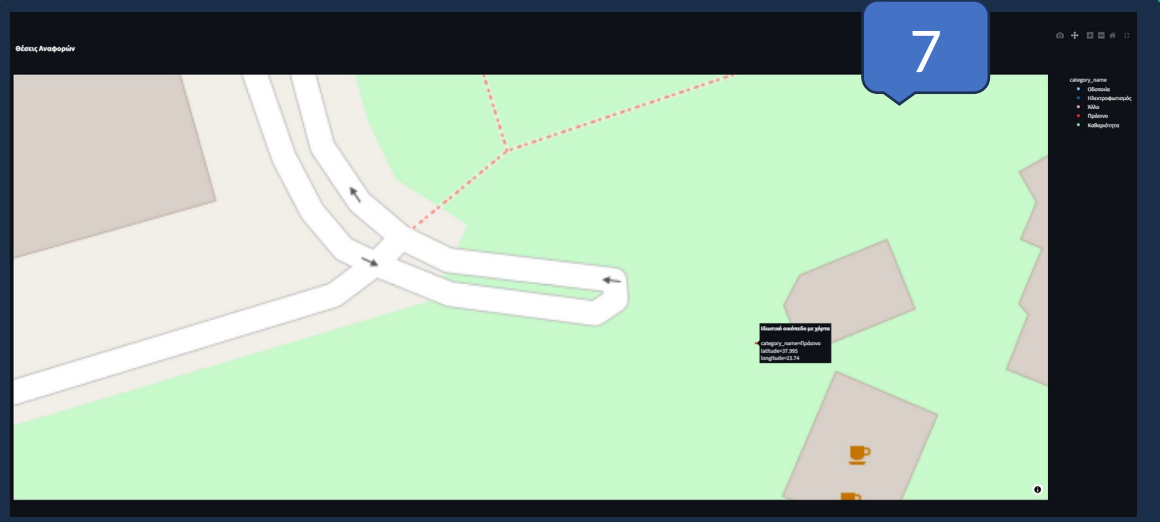
4

Αρχική σελίδα

Αναφορές - αναλυτική προβολή

Αποτελέσματα - Στατιστικά & API



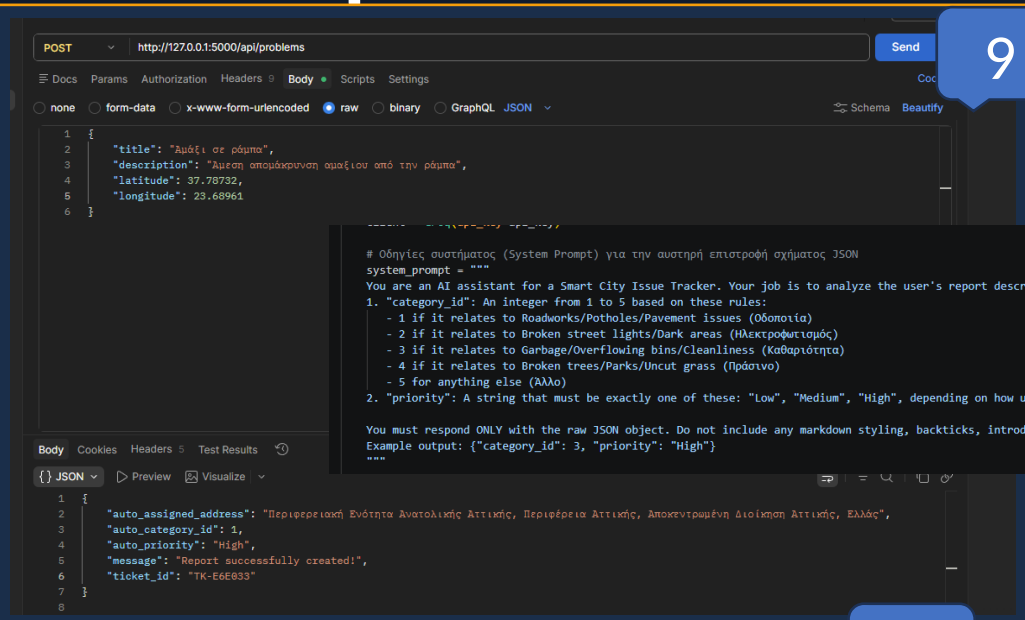






Αποτελέσματα - Στατιστικά & API

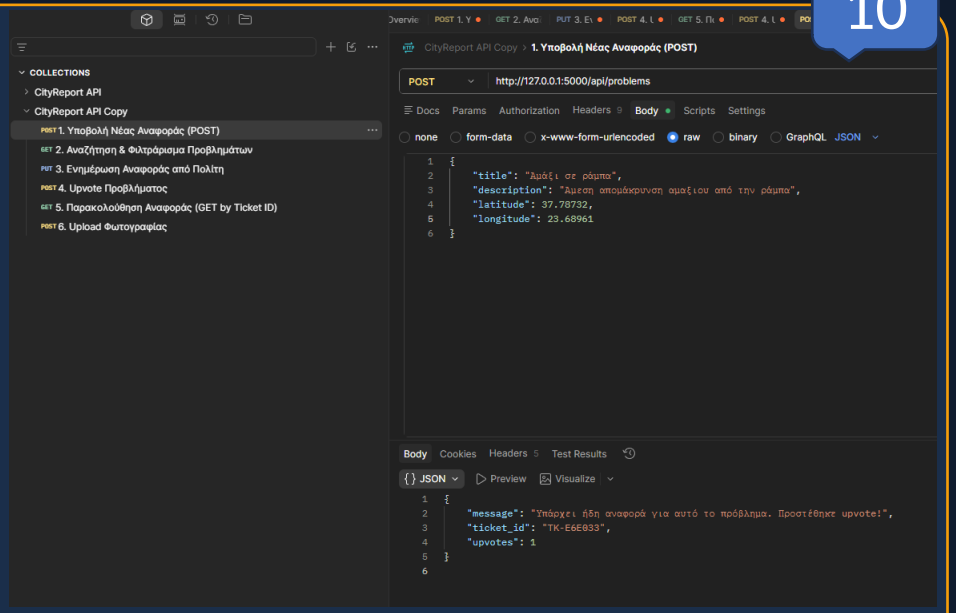
9



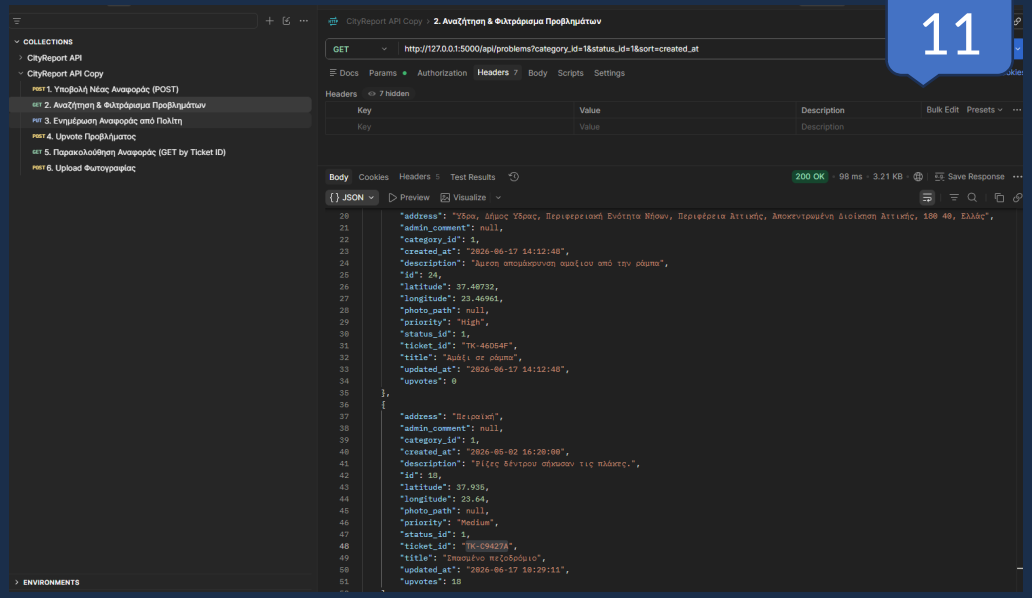

```
# Οδηγίες συστήματος (System Prompt) για την αυστηρή επιστροφή σχήματος JSON
system_prompt = """
You are an AI assistant for a Smart City Issue Tracker. Your job is to analyze the user's report description and return a JSON object with exactly two keys:
1. "category_id": An integer from 1 to 5 based on these rules:
   - 1 if it relates to Roadworks/Potholes/Pavement issues (Οδοποιία)
   - 2 if it relates to Broken street lights/Dark areas (Ηλεκτροφωτισμός)
   - 3 if it relates to Garbage/Overflowing bins/Cleanliness (Καθαριότητα)
   - 4 if it relates to Broken trees/Parks/Uncut grass (Πράσινο)
   - 5 for anything else (Άλλο)
2. "priority": A string that must be exactly one of these: "Low", "Medium", "High", depending on how urgent or dangerous the issue is to public safety.

You must respond ONLY with the raw JSON object. Do not include any markdown styling, backticks, introduction, or explanation.
Example output: {"category_id": 3, "priority": "High"}
"""
```

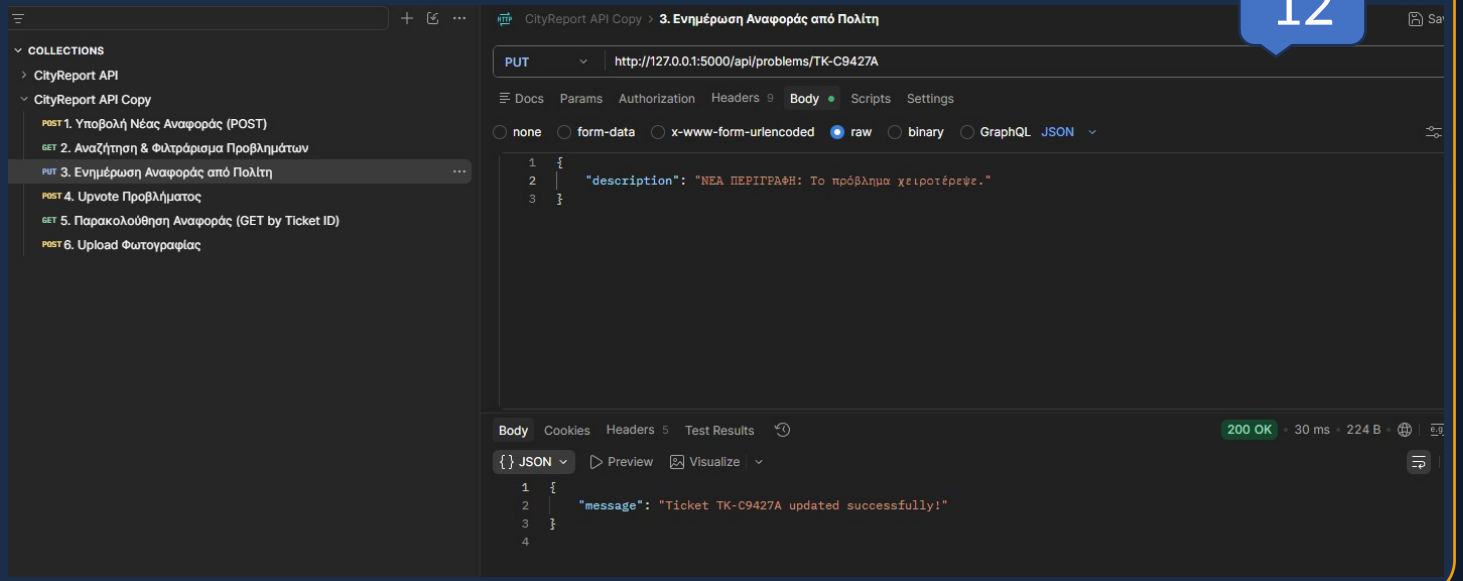
10



11

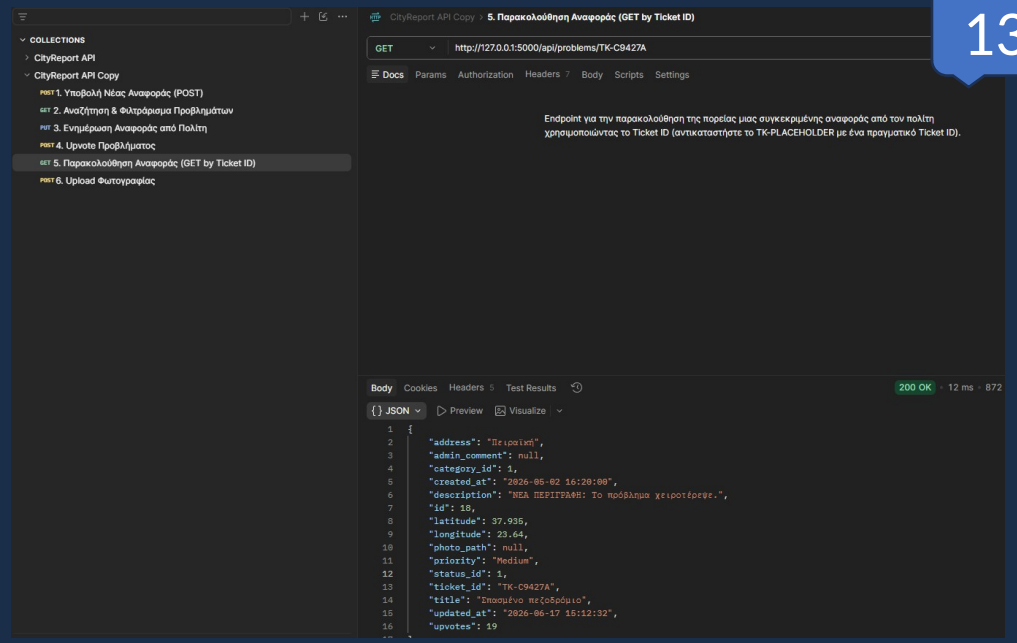


12

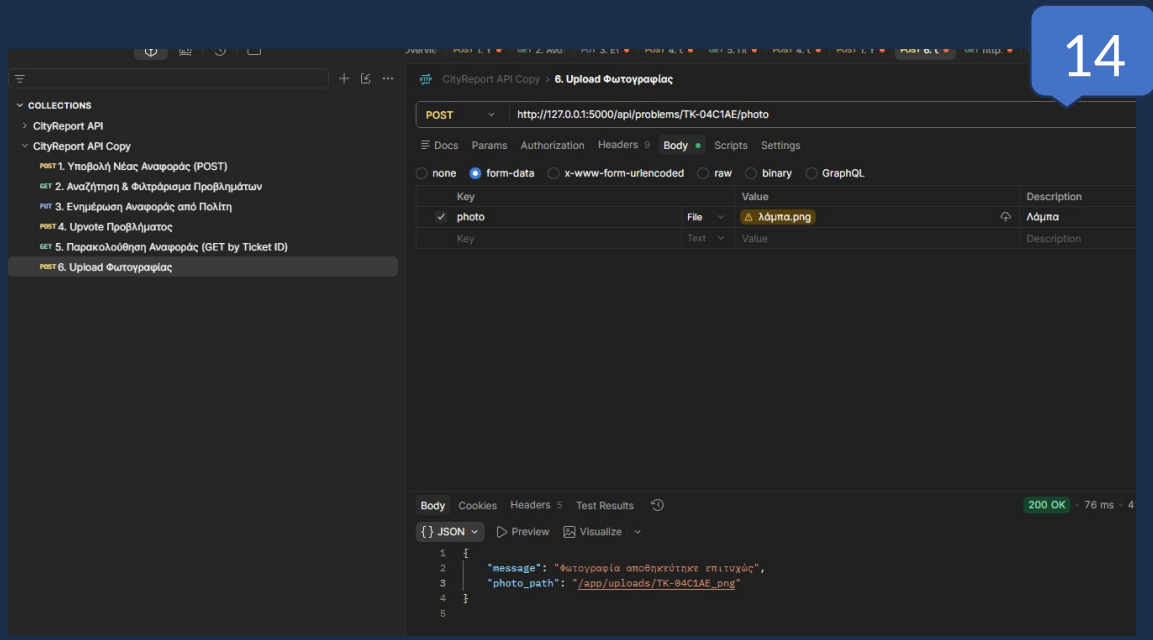


Αποτελέσματα - Στατιστικά & API

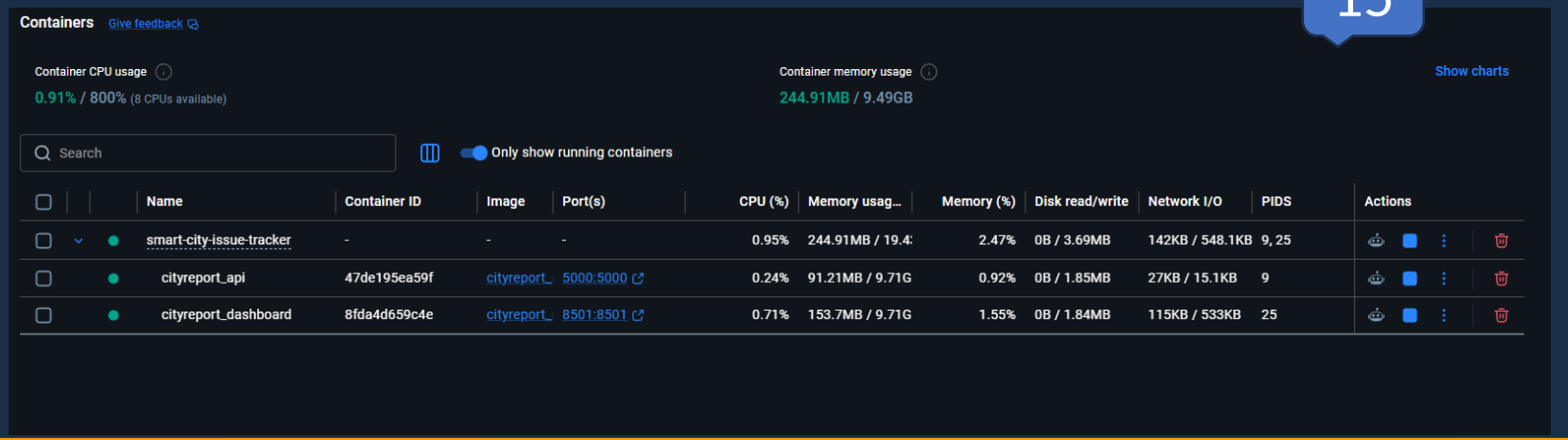
13



14



15



Τεχνικές Αποφάσεις & Βελτιώσεις

🔒 json.loads αντί eval() - ασφαλής ανάλυση AI απόκρισης

✅ Validation υποχρεωτικών πεδίων → HTTP 400 / 404

🔄 Αυτόματη ανίχνευση διπλότυπων → upvote αντί νέας εγγραφής

🤖 Fallback τιμές αν το Groq API δεν είναι διαθέσιμο

⚡ @st.cache_data - προσωρινή μνήμη για ταχύτερο dashboard

🐳 Docker: seed μόνο αν η βάση είναι κενή - δεν χάνονται δεδομένα

📁 secure_filename + έλεγχος τύπου αρχείου στο photo upload

📂 Αυτόματη δημιουργία φακέλου uploads/ κατά την εκκίνηση

🔑 Normalize ticket_id (.upper().strip()) σε όλα τα endpoints

Συμπεράσματα & Μελλοντικές Επεκτάσεις

Τι Επιτύχαμε

- Πλήρης κάλυψη απαιτήσεων
- REST API με 6 endpoints & AI κατηγοριοποίηση
- Πολυσέλιδο dashboard με στατιστικά & χάρτη
- 3 υποσυστήματα: AI, APIs, Ανάλυση
- Docker containerization με μία εντολή
- Δοκιμάστηκε τοπικά & σε containers

Μελλοντικές Επεκτάσεις

- Κινητή εφαρμογή / ιστοσελίδα για πολίτες
- Μετάβαση σε PostgreSQL
- Αυθεντικοποίηση & ρόλοι χρηστών
- Ειδοποιήσεις email/push στους πολίτες
- Χάρτης θερμότητας (heatmap) προβλημάτων
- Εμπλουτισμένη AI: σοβαρότητα & διπλότυπα

🏙️ CityReport

Σας Ευχαριστούμε!